



Prof. Dr. Thomas Wiemann

Robotik Übungsblatt 3

Sommersemester 2026

Aufgabe A3.1 (Bagfiles)

(25 Punkte)

ROS bietet die Möglichkeit Daten in sogenannte Bagfiles zu loggen¹. Fahren Sie mit dem Roboter eine Runde und speichern Sie die Odometrie-Daten in ein Bag-File. Spielen Sie anschließend das Bag-File ab und visualisieren Sie die geschätzte Roboterpose direkt in RViz. Dazu müssen Sie über "Add" einen Visualizer vom Typ "Odometry" hinzufügen. Zeigen Sie in einer Live-Demo, wie Sie die Daten aufnehmen, wiedergeben und anschließend visualisieren können.

Aufgabe A3.2 (IMU Positionsschätzung)

(25 Punkte)

Berechnen Sie aus den IMU-Messages die Position des Roboters. Dazu finden Sie im Repo nun das Launch-File `volksbot_imu.py`, das neben den Joystick jetzt auch die Phidgets-IMU des Roboters startet. Deren Daten werden auf dem Topic `imu/data_raw` gepublishet. Aus diesen können Sie den Betrag der mittleren Geschwindigkeit v approximieren, indem Sie zunächst die Komponenten v_x , v_y und v_z aus den gemeldeten Beschleunigungen und der Differenz Δ_t der Zeitstempel berechnen. Der benötigte Wert v ist dann die Länge des resultierenden Geschwindigkeitsvektors: $v = \sqrt{v_x^2 + v_y^2 + v_z^2}$. Den Drehwinkel θ bestimmen Sie direkt aus der Drehgeschwindigkeit um die z -Achse und Δ_t . Speichern Sie die berechneten x - und y -Koordinaten in eine Datei `imu.txt`. Vergleichen Sie die IMU-Trajektorie mit der Odometrie (am besten in einer Live-Demo oder mit einer ROS-Bag). Sie können dazu entweder auf Ihre Lösung des vorherigen Aufgabenblattes zurückgreifen oder die das in Moodle bereitgestellte Code-Gerüst verwenden.

Aufgabe B3.1 (Scanmatching)

(50 Punkte)

Gegeben sind Ihnen dazu zwei Laserscans auf den Topics `/model` und `/scan` im Bagfile `icp.bag`. Damit Sie sich die initiale Transformation zwischen beiden Scans ansehen können, publishen Sie eine statische `tf`-Transformation (zwischen `/model` und `/laser`) mit dem Kommando:

```
$ ros2 run tf2_ros static_transform_publisher 0 0 0 0 0 0 model laser
```

Bestimmen Sie die korrespondierenden Punkte zwischen beiden Mengen mittels der *Closest Point Heuristik* und visualisieren Sie diese als Linien mittels eines geeigneten `visualization_msgs/Marker`, die zwischen den Punktpaaren dargestellt werden. Berechnen Sie die Transformation zwischen den beiden Scans mittels ICP (*Iterative Closest Point*) und geben Sie Transformation und Fehler in jeder Iteration auf dem Terminal aus. Damit Sie sich den Fortschritt Ihrer ICP-Implementation gut in RViz ansehen können, sorgen Sie dafür, dass die Iteration im ICP mit einer Rate von z.B. 0.25 Hz abläuft und am Ende jeder Iteration die aktuelle Transformation über `tf` gepublisiert wird. *Hinweis:* Sie sollten den `static_transform_publisher` abschalten, sobald Sie selbst über Ihre ICP-Implementation Transformationen publishen. Ansonsten wird es zu Konflikten mit den kontinuierlich gepublizierten Transformationen kommen.

¹<https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Recording-And-Playing-Back-Data/Recording-And-Playing-Back-Data.html>